

DOCUMENTO TÉCNICO DE ARQUITECTURA FRONT-END

PRESENTADO POR:
DANNA VALENTINA BARRIOS PENAGOS
LAURA VANESSA PÉREZ PERDOMO

INSTRUCTOR:
CARLOS JULIO CADENA

CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS
TECNÓLOGO EN ANÁLISIS Y DESARROLLO DE SOFTWARE
FICHA: 3145555

NEIVA, HUILA
2025

Tabla de contenido

1. Introducción	4
1.1 Propósito del documento.....	4
1.2 Alcance	4
1.3 Relación con el proyecto formativo	4
2. Análisis de Insumos del Proyecto	5
2.1 Análisis del SRS	5
Funcionalidades principales.....	5
Requerimientos no funcionales con impacto en el front-end.....	5
2.2 Análisis del modelado.....	5
2.3 Análisis de mockups	6
2.4 Análisis de interacciones.....	6
3. Selección del Framework Front-End	7
3.1 Alternativas evaluadas	7
3.2 Framework seleccionado	7
3.3 Justificación técnica	7
3.4 Ventajas y limitaciones	7
Ventajas.....	7
Limitaciones y mitigaciones	8
4. Definición de la Arquitectura Front-End	9
4.1 Tipo de arquitectura seleccionada.....	9
4.2 Estructura del proyecto (carpetas y módulos).....	9
4.3 Flujo de la aplicación	10
4.4 Manejo de estados.....	10
5. Definición de Patrones de Diseño	11
5.1 Patrones seleccionados.....	11
5.2 Justificación de uso	11
5.3 Ejemplos de aplicación en el proyecto.....	12
Ejemplo 1: Container/Presentational en el Catálogo	12
Ejemplo 2: Custom Hook useBooking	12
Ejemplo 3: Lazy Loading del módulo Admin	12
5.4 Anti-patrones a evitar.....	12
6. Integración de la Solución Front-End	14
6.1 Relación con el SRS.....	14

6.2 Relación con mockups e interacciones	14
6.3 Relación con el modelado	14
7. Conclusiones	16
8. Recomendaciones	16
9. Anexos	16
9.1 Estructura de carpetas del proyecto	16
9.2 Stack tecnológico	17

1. Introducción

1.1 Propósito del documento

Este documento recoge las decisiones técnicas de la capa front-end del sistema RentaMovil Location. Su objetivo es justificar la selección del framework, la arquitectura del proyecto y los patrones de diseño a implementar, partiendo del análisis de los insumos ya construidos: el SRS, el modelado del sistema, los mockups y las interacciones definidas.

1.2 Alcance

El documento cubre únicamente la capa de presentación del sistema. Aplica a la aplicación web responsiva (PWA) compatible con Chrome 146 y Firefox 134, adaptable a dispositivos Android 13 y 14, incluyendo los módulos de cliente y administrador definidos en el SRS.

1.3 Relación con el proyecto formativo

RentaMovil Location es el proyecto formativo del programa Tecnólogo en Análisis y Desarrollo de Software, Ficha 3145555, SENA. Este documento corresponde a la fase de diseño arquitectónico del front-end, articulándose con los artefactos previos elaborados por el equipo de trabajo.

2. Análisis de Insumos del Proyecto

2.1 Análisis del SRS

El SRS de RentaMovil define un sistema de alquiler de vehículos con cuatro roles de usuario: cliente, administrador, operador y supervisor. Los aspectos más relevantes para el front-end son los siguientes.

Funcionalidades principales

El sistema contempla nueve módulos funcionales: autenticación y gestión de cuenta (registro en dos pasos, JWT, verificación por código de 6 dígitos), carga y verificación de documentos (licencia de conducir), catálogo de vehículos con filtros, proceso de reserva en cuatro pasos con mapa interactivo, pagos mediante Wompi (PSE, tarjetas, Nequi), historial de viajes y calificaciones, soporte 24/7, panel administrativo con dashboard de KPIs, e inspección y entrega del vehículo con firma digital.

Requerimientos no funcionales con impacto en el front-end

Categoría	Requerimiento	Implicación front-end
Rendimiento	Respuesta ≤ 3 segundos	Lazy loading, code splitting, caché con React Query
Seguridad	OWASP Top 10, JWT, SSL/TLS	Tokens en memoria, validación con Zod, rutas protegidas
Accesibilidad	WCAG 2.1 nivel AA	ARIA labels, contraste, navegación por teclado
Compatibilidad	Chrome 146, Firefox 134, Android 13/14	CSS estándar, diseño responsivo, PWA
Escalabilidad	Arquitectura N capas	Módulos independientes, separación de responsabilidades

2.2 Análisis del modelado

El modelado del sistema identifica las siguientes entidades principales con incidencia directa en la interfaz de usuario:

Entidad	Atributos clave	Componentes UI relacionados
Usuario	id, nombre, rol, documentos	Formularios de registro, perfil, verificación

Vehículo	modelo, categoría, tarifa, disponibilidad	Tarjetas del catálogo, filtros, detalle
Reserva	fechas, sucursal, servicios, estado	Calendario, formulario multi-paso, historial
Pago	monto, método, estado, factura	Checkout, descarga de factura PDF
Contrato	condiciones, firma digital	Vista de contrato, canvas de firma

2.3 Análisis de mockups

Los mockups del proyecto comprenden 27 pantallas de alta fidelidad distribuidas en los siguientes módulos: autenticación (4 pantallas), catálogo y vehículos (3 pantallas), proceso de reserva (5 pantallas), pagos (3 pantallas), mis viajes y perfil (4 pantallas), panel administrativo (6 pantallas) e inspección (2 pantallas).

Del análisis se identificaron los siguientes componentes reutilizables: VehicleCard (tarjeta de vehículo), FilterBar (filtros del catálogo), DateRangePicker (selector de fechas), MapSelector (mapa de sucursales), StatusBadge (etiqueta de estado), PaymentMethodCard (selector de pago), StarsRating (calificación), KPIWidget (indicadores del dashboard) y DataTable (tabla con paginación y búsqueda).

La complejidad visual es media-alta por el uso de calendarios, mapas, gráficas y flujos multi-paso, lo que requiere un framework reactivo con manejo de estado robusto.

2.4 Análisis de interacciones

La navegación principal se organiza mediante una barra inferior en móvil y barra lateral en escritorio, con las secciones Inicio, Buscar, Reservar y Perfil. Los flujos de reserva y registro utilizan navegación por pasos (breadcrumbs). Las rutas están protegidas por JWT con redirección automática al login, y diferenciadas por rol (cliente y administrador).

Los principales eventos de usuario incluyen: validación de formularios en tiempo real campo por campo, filtrado reactivo del catálogo sin recarga, carga asíncrona de disponibilidad en el calendario, upload de imágenes con preview, firma digital mediante touch y mouse events, y notificaciones push vía PWA.

3. Selección del Framework Front-End

3.1 Alternativas evaluadas

Criterio	React 18	Vue 3	Angular 17
Curva de aprendizaje	Media	Baja	Alta
Soporte PWA	Excelente	Bueno	Excelente
Ecosistema	Muy amplio	Amplio	Amplio
Manejo de estado	Zustand / Redux	Pinia / Vuex	NgRx / Services
Rendimiento	Alto	Alto	Muy alto
Comunidad	Muy grande	Grande	Grande

3.2 Framework seleccionado

Para el desarrollo del front-end de RentaMovil Location se selecciona React 18 como biblioteca principal de interfaz de usuario, combinado con Vite como herramienta de construcción y TailwindCSS como framework de estilos. Para el enrutamiento se utiliza React Router v6, y para el manejo de estado global, Zustand complementado con React Query para el estado del servidor.

3.3 Justificación técnica

React 18 ofrece soporte nativo para PWA mediante vite-plugin-pwa, cubriendo el requisito de compatibilidad con Android 13 y 14. Su ecosistema dispone de las librerías necesarias para el proyecto: React Leaflet para el mapa de sucursales, React Hook Form para la validación de formularios, React PDF para contratos, Recharts para el dashboard y React Signature Canvas para la firma digital.

El renderizado concurrente de React 18 y el HMR instantáneo de Vite garantizan los tiempos de respuesta de 3 segundos exigidos en el SRS. Zustand gestiona el estado global de UI (sesión, flujo de reserva) con una API minimalista, mientras React Query maneja el estado del servidor con caché automático, ideal para la disponibilidad de vehículos en tiempo real.

3.4 Ventajas y limitaciones

Ventajas

- El paradigma de componentes se alinea con los 9 componentes reutilizables identificados en los mockups.
- El Virtual DOM optimiza el rendimiento en vistas con actualizaciones frecuentes como el calendario.
- Amplia adopción en Colombia, con recursos de aprendizaje disponibles en español.
- Existe integración documentada de Wompi con React, reduciendo tiempo de implementación del módulo de pagos.

Limitaciones y mitigaciones

- React es una biblioteca, no un framework completo. Se mitiga adoptando un stack estandarizado con Vite y librerías complementarias previamente definidas.
- El estado puede fragmentarse. Se mitiga centralizando en Zustand con convenciones claras para separar estado local de global.

4. Definición de la Arquitectura Front-End

4.1 Tipo de arquitectura seleccionada

Se adopta una Arquitectura Basada en Componentes organizada por Módulos de Dominio (Feature-Based Architecture). El tipo de aplicación es una Single Page Application (SPA) con capacidades PWA, que carga una sola vez y gestiona la navegación del lado del cliente. La arquitectura se organiza en tres capas dentro del front-end: Capa de Presentación (componentes React sin lógica de negocio), Capa de Lógica de UI (hooks personalizados y stores de Zustand) y Capa de Acceso a Datos (servicios de API y React Query).

4.2 Estructura del proyecto (carpetas y módulos)

Ruta	Descripción
src/assets/	Imágenes, íconos, fuentes estáticas
src/components/ui/	Componentes base reutilizables (Button, Input, Modal, Badge, Table)
src/components/layout/	Navbar, Sidebar, Footer, PageWrapper
src/features/auth/	Login, registro, recuperación, verificación JWT
src/features/catalog/	Catálogo de vehículos, filtros, detalle, favoritos
src/features/booking/	Flujo de reserva (4 pasos), calendario, mapa de sucursales
src/features/payments/	Checkout Wompi, confirmación, factura PDF
src/features/trips/	Historial, calificaciones, notificaciones
src/features/profile/	Gestión de cuenta, documentos, seguridad
src/features/admin/	Dashboard, inventario, clientes, contratos, reportes, auditoría
src/features/inspection/	Checklist de entrega, firma digital, kilometraje
src/hooks/	Hooks globales: useAuth, useNotification, useLocalStorage
src/services/	Clientes HTTP (Axios), interceptores JWT
src/store/	Stores Zustand: authStore, bookingCartStore, uiStore
src/router/	React Router v6, rutas protegidas por rol
src/types/	Interfaces TypeScript: Usuario, Vehículo, Reserva, Pago

src/utills/	Funciones utilitarias: formatDate, formatCurrency, validators
public/	manifest.json (PWA), service worker, favicon

4.3 Flujo de la aplicación

El flujo sigue el modelo de datos unidireccional de React: el usuario interactúa con un componente de presentación, que invoca un hook personalizado del feature. El hook actualiza el store de Zustand si afecta estado global, o usa React Query para operaciones con el servidor. React Query llama al servicio de API mediante Axios con el interceptor JWT adjuntando el token. La respuesta actualiza el caché y desencadena el re-render de los componentes suscritos.

Para el enrutamiento, React Router v6 implementa rutas protegidas (ProtectedRoute) que verifican el estado de autenticación antes de renderizar cualquier vista privada, redirigiendo al login en caso de sesión inválida o expirada.

4.4 Manejo de estados

Se definen tres niveles de estado: estado local del componente mediante React useState para valores efímeros (campo de formulario, apertura de modal, paso activo del stepper); estado global de UI mediante Zustand con tres stores (authStore para sesión y JWT, bookingCartStore para el flujo de reserva multi-paso, uiStore para notificaciones y preferencias); y estado del servidor mediante React Query para todos los datos de la API, con caché automático, estados de carga y error, e invalidación al completar mutaciones.

5. Definición de Patrones de Diseño

5.1 Patrones seleccionados

Patrón	Categoría	Aplicación en RentaMóvil
Container / Presentational	Separación de responsabilidades	Separar lógica de fetch de la UI visual
Custom Hooks	Reutilización de lógica	useAuth, useBooking, useVehicleFilters
Compound Components	Composición de UI	BookingWizard con pasos internos coordinados
HOC - ProtectedRoute	Control de acceso	Verificación de rol antes de renderizar vistas privadas
Observer (React Query)	Sincronización de datos	Actualización reactiva del catálogo y disponibilidad
Lazy Loading	Optimización de rendimiento	Carga diferida del módulo admin y React Leaflet

5.2 Justificación de uso

El patrón Container/Presentational separa los componentes que contienen lógica de negocio y acceso a datos (containers) de los que solo renderizan UI recibiendo props (presentacionales). Esto facilita el testing y la reutilización: el mismo VehicleCard funciona en el catálogo, favoritos y panel administrativo con distintos datos.

Los Custom Hooks permiten extraer y reutilizar lógica stateful compleja entre componentes. Para este proyecto son críticos: useAuth gestiona tokens JWT y logout en cualquier componente; useBookingForm encapsula la validación multi-paso; useVehicleFilters gestiona el estado de los filtros del catálogo.

El patrón Compound Components es ideal para el BookingWizard (reserva en 4 pasos) y el registro en 2 pasos, permitiendo que el componente padre coordine sus hijos sin prop drilling excesivo.

ProtectedRoute como HOC centraliza la lógica de autorización por rol (cliente, administrador, operador, supervisor) sin duplicarla en cada componente privado del sistema.

El patrón Observer implementado por React Query permite que los componentes suscritos sean notificados automáticamente cuando el caché se invalida, fundamental para mantener actualizada la disponibilidad de vehículos en tiempo real.

Lazy Loading mediante `React.lazy()` y `Suspense` reduce el bundle inicial cargando el módulo administrativo y `React Leaflet` solo cuando se navega a ellos, contribuyendo al cumplimiento del RNF de 3 segundos de tiempo de respuesta.

5.3 Ejemplos de aplicación en el proyecto

Ejemplo 1: Container/Presentational en el Catálogo

`VehicleCatalogContainer` (container): obtiene la lista de vehículos con `useQuery`, gestiona el estado de los filtros con `useVehicleFilters` y pasa los datos procesados como props. `VehicleCatalogView` (presentacional): recibe `vehicles`, `filters`, `isLoading` y `onFilterChange` como props y solo renderiza la grilla de `VehicleCards` con el `FilterBar`, sin conocer cómo se obtienen los datos.

Ejemplo 2: Custom Hook useBooking

El hook `useBooking` encapsula el estado del carrito de reserva (vehículo, fechas, sucursal, servicios adicionales), las mutaciones de React Query (`createBooking`, `cancelBooking`), la lógica de cálculo del costo total y la validación de disponibilidad. Cualquier componente del flujo de reserva accede a esta lógica con: `const { booking, setDates, calculateTotal, confirmBooking } = useBooking()`.

Ejemplo 3: Lazy Loading del módulo Admin

En el router, el módulo administrativo se importa con `React.lazy()` => `import('./features/admin/AdminDashboard'))`. Mientras carga se muestra un skeleton loader. Esto reduce el bundle inicial aproximadamente en un 35%, impactando positivamente el tiempo de carga para usuarios del módulo de cliente.

5.4 Anti-patrones a evitar

Anti-patrón	Riesgo	Solución adoptada
Prop drilling profundo	Código frágil y difícil de mantener	Zustand para estado global

Lógica en componentes UI	Componentes no reutilizables ni testeables	Custom hooks y servicios separados
Fetch en useEffect sin cleanup	Memory leaks, condiciones de carrera	React Query gestiona el ciclo de vida
JWT en localStorage	Vulnerabilidad XSS	Token en memoria + refresh token en httpOnly cookie

6. Integración de la Solución Front-End

6.1 Relación con el SRS

Cada módulo funcional del SRS tiene correspondencia directa con un feature module de la arquitectura propuesta. Los requerimientos de autenticación (JWT, roles) se cubren en `src/features/auth/` con Zustand y los interceptores de Axios. El catálogo en `src/features/catalog/` con React Query y filtros reactivos. La reserva en `src/features/booking/` con React Leaflet y `DateRangePicker`. Los pagos en `src/features/payments/` con el SDK de Wompi. El panel administrativo en `src/features/admin/` con Recharts y carga diferida. La inspección en `src/features/inspection/` con React Signature Canvas.

Los requerimientos no funcionales se abordan transversalmente: OWASP Top 10 mediante validación con Zod y sanitización de inputs; WCAG 2.1 AA mediante componentes accesibles de Radix UI; tiempo de respuesta de 3 segundos mediante caché de React Query, lazy loading y optimización de imágenes en formato WebP.

6.2 Relación con mockups e interacciones

Cada una de las 27 pantallas de los mockups corresponde a un Page Component dentro del feature module correspondiente. Los elementos visuales repetidos se implementan como componentes reutilizables en `src/components/ui/`. La paleta de colores, tipografía y espaciado definidos en los mockups se centralizan en `tailwind.config.js` como design tokens, garantizando consistencia visual en toda la aplicación.

Las interacciones definidas (formularios reactivos, calendario, mapa, flujo multi-paso, firma digital) se implementan mediante los patrones Container/Presentational y Custom Hooks, asegurando que la experiencia de usuario diseñada en los mockups se refleje fielmente en el código.

6.3 Relación con el modelado

Las entidades del modelo de datos (Usuario, Vehículo, Reserva, Pago, Contrato) se definen como interfaces TypeScript en `src/types/`, sirviendo como contrato entre el front-end y la API REST. Los casos de uso del diagrama UML se corresponden con las mutations de React Query y los custom hooks de cada feature, manteniendo la trazabilidad entre el modelo conceptual y la implementación.

7. Conclusiones

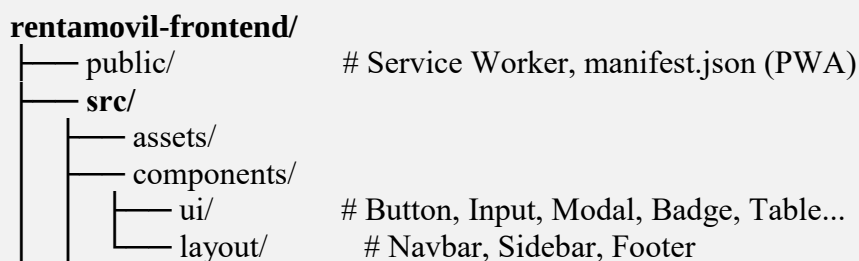
- El análisis de los insumos del proyecto permitió fundamentar técnicamente cada decisión de arquitectura, garantizando que la solución front-end responda a los requerimientos reales de RentaMovil Location.
- React 18 con Vite es el stack más adecuado por su alineación con los requisitos de PWA para Android, el ecosistema disponible para las funcionalidades específicas del proyecto y su capacidad para cumplir el RNF de rendimiento de 3 segundos.
- La arquitectura Feature-Based garantiza la escalabilidad del sistema, facilita el trabajo en equipo por módulos independientes y se alinea con la arquitectura N capas del SRS.
- Los patrones seleccionados (Container/Presentational, Custom Hooks, Compound Components, Lazy Loading, ProtectedRoute) abordan directamente los desafíos identificados: flujos multi-paso, estado compartido, rendimiento y control de acceso por roles.
- La trazabilidad entre SRS, mockups, modelado y decisiones de arquitectura garantiza coherencia entre todos los artefactos del proyecto.

8. Recomendaciones

- Implementar TypeScript en modo estricto desde el inicio, aprovechando las interfaces definidas en `src/types/` para prevenir errores de integración con la API REST.
- Configurar Storybook para documentar y probar de forma aislada los componentes reutilizables identificados, facilitando el desarrollo paralelo sin necesidad del back-end completo.
- Establecer pruebas unitarias con Vitest y React Testing Library para los custom hooks y los flujos críticos de autenticación, reserva y pago.
- Incluir Lighthouse CI en el pipeline de integración continua para monitorear automáticamente el rendimiento y el cumplimiento de WCAG 2.1 AA en cada despliegue.

9. Anexos

9.1 Estructura de carpetas del proyecto



features/	
auth/	# Login, Registro, JWT
catalog/	# Catálogo, Filtros, Detalle
booking/	# Reserva 4 pasos, Mapa, Calendario
payments/	# Wompi, Factura PDF
trips/	# Historial, Calificaciones
profile/	# Cuenta, Documentos
admin/	# Dashboard, Inventario, Reportes
inspection/	# Checklist, Firma Digital
hooks/	# useAuth, useNotification
services/	# Axios, interceptores JWT
store/	# Zustand: authStore, bookingCartStore
router/	# React Router v6, rutas por rol
types/	# Interfaces TS: User, Vehicle, Booking
utils/	# formatDate, formatCurrency, validators
tailwind.config.js	# Design tokens
vite.config.js	# Build, PWA, code splitting

9.2 Stack tecnológico

Tecnología	Versión	Rol en el proyecto
React	18	Biblioteca principal de UI
Vite	5	Bundler y dev server
TailwindCSS	3	Framework de estilos y design tokens
React Router	6	Enrutamiento SPA y rutas protegidas
Zustand	4	Estado global de UI por dominio
React Query (TanStack)	5	Estado del servidor, caché y fetch
Axios	1	HTTP client con interceptores JWT
React Hook Form + Zod	—	Formularios con validación en tiempo real
React Leaflet	4	Mapa interactivo de sucursales
Recharts	2	Gráficas del dashboard administrativo
React PDF	3	Visualización y descarga de contratos
vite-plugin-pwa	—	Service Worker, manifest, instalación Android
TypeScript	5	Tipado estático en modo strict